PERFORMANCE EVALUATION OF DROPOUT ON FULLY CONNECTED LAYERS

&

BATCH NORMALIZATION ON CONVOLUTIONAL LAYERS

DATE: MAY 14, 2021

PROJECT: TYPE A

NAME: KAUSTUBH NATHURAM JADHAV

NETID: knj25

RUID: 196003155

1. Abstract

The effects of batch normalization and dropout layers on the output of Convolutional Neural Networks will be discussed in this article. The Lenet-5 model was used as the architecture/model in the convolutional neural network for this experiment, and the dataset used was the MNIST dataset. We will also go through basic principles of CNN and Lenet-5 architecture, as well as experimental findings and interpretation, before concluding.

2. Introduction (Descriptions)

a) Convolutional Neural Network
b) Batch Normalization
c) Dropout

a) Convolutional Neural Network

Convolutional neural networks are composed of multiple layers of artificial neurons. Artificial neurons, a rough imitation of their biological counterparts, are mathematical functions that calculate the weighted sum of multiple inputs and outputs an activation value. The behavior of each neuron is defined by its weights. [1]

When fed with the pixel values, the artificial neurons of a CNN pick out various visual features. When you input an image into a ConvNet, each of its layers generates several activation maps. Activation maps highlight the relevant features of the image. Each of the neurons takes a patch of pixels as input, multiplies their color values by its weights, sums them up, and runs them through the activation function. The first (or bottom) layer of the CNN usually detects basic features such as horizontal, vertical, and diagonal edges. The output of the first layer is fed as input of the next layer, which extracts more complex features, such as corners and combinations of edges. As you move deeper into the convolutional neural network, the layers start detecting higher-level features such as objects, faces, and more. [1]

The operation of multiplying pixel values by weights and summing them is called “convolution” (hence the name convolutional neural network). A CNN is usually composed of several convolution layers, but it also contains other components. The final layer of a CNN is a

classification layer, which takes the output of the final convolution layer as input (remember, the higher convolution layers detect complex objects). [1]

b) Batch Normalization

Batch normalization is a technique for training very deep neural networks that standardizes the inputs to a layer for each mini batch. This has the effect of stabilizing the learning process and dramatically reducing the number of training epochs required to train deep networks. [3]

Batch Normalization - a data pre-processing technique allows us to use much higher learning rates. It reduces strong dependency on initialization. It also acts as a form of regularizer and in some cases eliminates the need for Dropout. In a neural network, batch normalization is achieved through a normalization step that fixes the means and variances of each layer's inputs. Ideally, the normalization would be conducted over the entire training set, but to use this step jointly with stochastic optimization methods, it is impractical to use the global information. Thus, normalization is restrained to each mini batch in the training process.

Batch normalization can be implemented during training by calculating the mean and standard deviation of each input variable to a layer per mini-batch and using these statistics to perform the standardization. Alternately, a running average of mean and standard deviation can be maintained across mini batches but may result in unstable training. [3]

c) Dropout

A fully connected layer occupies most of the parameters, and hence, neurons develop co-dependency amongst each other during training which curbs the individual power of each neuron leading to over-fitting of training data. Regularization reduces overfitting by adding a penalty to the loss function. By adding this penalty, the model is trained such that it does not learn an interdependent set of features weights. Those of you who know Logistic Regression might be familiar with L1 (Laplacian) and L2 (Gaussian) penalties. Dropout is an approach to regularization in neural networks which helps reduce interdependent learning amongst the neurons. [2]

Dropout works mainly because of below reasons:

1. If a hidden unit is always trained with another hidden unit, they will co-adapt to the training data.
2. If a hidden unit has to work well with a different combination of other hidden neurons, it will optimize itself to be useful.

3. Each time we present a training batch, we randomly omit each hidden unit with probability p.

3. Lenet – 5 Models

In this section we will discuss the general configurations and architectures of different variations of Lenet-5 models mentioned in problem statement for experiments as listed below:

1. Lenet-5 Model with Dropout in Fully Connected Layer and Batch Normalization in Convolutional Layer
2. Lenet-5 Model with only Dropout in Fully Connected Layer
3. Lenet-5 Model with only Batch Normalization in Convolutional Layer
4. Simple/Standard Lenet-5 Model with NO Batch Normalization or Dropout layers

3.1. General Configuration

- I used the NN Sequential Module of Pytorch for training and testing the model.
- Also, I used Cross Entropy Loss Layer in model training as Pytorch Cross Entropy Layer calculates the Softmax as well as the loss simultaneously. Also, since this problem was **MULTICLASS CLASSIFICATION** problem, I had used this layer while training for calculating loss.
- The number of epochs is 25 as after 8-10 epochs accuracy goes to 99%. So there was no point in going up to 50 epochs for training the model.
- The batch size considered here for training images is 128 while for testing images is 100.
- The training dataset is randomly shuffled while the testing dataset is not shuffled.
- Learning rate is 0.05, momentum is 0.9 and weight decay is 5e-4
- Also, I have used SGD optimizer for optimizing weights after backward propagation

3.2. Dropout in FC and Batch Normalization in CONV

As per first part of the problem statement, the architecture of Lenet-5 is:

- Starting layer is Convolution2d layer with input of 1 channel and output as 6 channels with Kernel filter size as 5,5 followed by Relu function **– c1 and relu1**
- Now, I normalize the convolved data using batch normalization layer for 6 channels **- bn1**

● Then I perform sub-sampling using MaxPool2d with kernel size as 2,2 and stride of 2 **– s2**

● Further, then comes Convolution2d layer with input as 6 channels and outputs as 16 channels with Kernel filter size as 5,5 followed by Relu function **– c3 and relu3**

● Again, I perform sub-sampling using MaxPool2d with kernel size as 2,2 and stride of 2 **– s4**

● In the last convolution network, I have Convolution2d layer with input as 16 channels and outputs as 120 channels with Kernel filter size as 4,4 followed by Relu function **– c5, relu5**

● Now very important task – I have to flatten the images i.e. output from convolution network so that they can be fed into fully connected layers. Thus, I have flattened the output of convolution network using torch.view() function in forward pass.

● Then, once data is flattened, I fed it to fully connected Linear layer with input as 120 and output as 84 followed by Relu function **– f6 and relu6**

● Now, I have passed data through dropout layer with rate as 0.5 (widely used) - **drop6**

● Further, I again pass the data to Linear Layer with input as 84 and final output as 10 **– f7**

● This output is passed to final LogSoftmax layer **– sig7**

3.3. Dropout in FC

As per second part of the problem statement, the architecture of Lenet-5 is:

● Starting layer is Convolution2d layer with input of 1 channel and output as 6 channels with Kernel filter size as 5,5 followed by Relu function **– c1 and relu1**

● Then I perform sub-sampling using MaxPool2d with kernel size as 2,2 and stride of 2 **– s2**

● Further, then comes Convolution2d layer with input as 6 channels and outputs as 16 channels with Kernel filter size as 5,5 followed by Relu function **– c3 and relu3**

● Again, I perform sub-sampling using MaxPool2d with kernel size as 2,2 and stride of 2 **–s4**

● In the last convolution network, I have Convolution2d layer with input as 16 channels and outputs as 120 channels with Kernel filter size as 4,4 followed by Relu function **– c5, relu5**

● Now a very important task – I have to flatten the images i.e. output from convolution network so that they can be fed into fully connected layers. Thus, I have flattened the output of convolution network using torch.view() function in forward pass.

● Then, once data is flattened, I fed it to fully connected Linear layer with input as 120 and output as 84 followed by Relu function **– f6 and relu6**

● Now, I have passed data through dropout layer with rate as 0.5 (widely used) - **drop6**

● Further, I again pass the data to Linear Layer with input as 84 and final output as 10 **– f7**

● This output is passed to final LogSoftmax layer **– sig7**

3.4. Batch Normalization in CONV

As per third part of the problem statement, the architecture of Lenet-5 is:

● Starting layer is Convolution2d layer with input of 1 channel and output as 6 channels with Kernel filter size as 5,5 followed by Relu function **– c1 and relu1**

● Now, I normalize the convolved data using batch normalization layer for 6 channels **- bn1**

● Then I perform sub-sampling using MaxPool2d with kernel size as 2,2 and stride of 2 **– s2**

● Further, then comes Convolution2d layer with input as 6 channels and outputs as 16 channels with Kernel filter size as 5,5 followed by Relu function **– c3 and relu3**

● Again, I perform sub-sampling using MaxPool2d with kernel size as 2,2 and stride of 2 **–s4**

● In the last convolution network, I have Convolution2d layer with input as 16 channels and outputs as 120 channels with Kernel filter size as 4,4 followed by Relu function **– c5, relu5**

● Now a very important task – I have to flatten the images i.e. output from convolution network so that they can be fed into fully connected layers. Thus, I have flattened the output of convolution network using torch.view() function in forward pass.

● Then, once data is flattened, I fed it to fully connected Linear layer with input as 120 and output as 84 followed by Relu function **– f6 and relu6**

● Further, I again pass the data to Linear Layer with input as 84 and final output as 10 **– f7**

● This output is passed to final LogSoftmax layer **– sig7**

3.5. No Dropout in FC and No Batch Normalization in CONV

As per last/fourth part of the problem statement, the architecture of Lenet-5 is:

● Starting layer is Convolution2d layer with input of 1 channel and output as 6 channels with Kernel filter size as 5,5 followed by Relu function **– c1 and relu1**

● Then I perform sub-sampling using MaxPool2d with kernel size as 2,2 and stride of 2 **– s2**

● Further, then comes Convolution2d layer with input as 6 channels and outputs as 16 channels with Kernel filter size as 5,5 followed by Relu function **– c3 and relu3**

● Again, I perform sub-sampling using MaxPool2d with kernel size as 2,2 and stride of 2 **– s4**

● In the last convolution network, I have Convolution2d layer with input as 16 channels and outputs as 120 channels with Kernel filter size as 4,4 followed by Relu function **– c5, relu5**

● Now a very important task – I have to flatten the images i.e. output from convolution network so that they can be fed into fully connected layers. Thus, I have flattened the output of convolution network using torch.view() function in forward pass.

● Then, once data is flattened, I fed it to fully connected Linear layer with input as 120 and output as 84 followed by Relu function **– f6 and relu6**

● Further, I again pass the data to Linear Layer with input as 84 and final output as 10 **– f7**

● This output is passed to final LogSoftmax layer **– sig7**

This is the simple Lenet-5 model where **NO** dropout layer and **NO** batch normalization. layers are present in FC and CONV layers.

4. Results

All the models were compiled on **Google Colab**.

Lenet Architecture	Total time
	Seconds
With DO and BN	570.42
With DO	362.655
With BN	864.222
No DO and BN	452.57

4.1. Discussion

The performance accuracy for all variations of Lenet models is 99%.

4.1.1. Dropout in FC and Batch Normalization in CONV (File: **lenet_mnist_DO_BN.py**)

```
Test set: Average loss: 0.0379, Accuracy: 9897/10000 (99%)

Train Epoch: 25 [0/60000 (0%)]  Loss: 0.049760
Train Epoch: 25 [1280/60000 (2%)]       Loss: 0.034144
Train Epoch: 25 [2560/60000 (4%)]       Loss: 0.077132
Train Epoch: 25 [3840/60000 (6%)]       Loss: 0.009070
Train Epoch: 25 [5120/60000 (9%)]       Loss: 0.032414
Train Epoch: 25 [6400/60000 (11%)]      Loss: 0.068235
Train Epoch: 25 [7680/60000 (13%)]      Loss: 0.040402
Train Epoch: 25 [8960/60000 (15%)]      Loss: 0.015429
Train Epoch: 25 [10240/60000 (17%)]     Loss: 0.027751
Train Epoch: 25 [11520/60000 (19%)]     Loss: 0.007856
Train Epoch: 25 [12800/60000 (21%)]     Loss: 0.003951
Train Epoch: 25 [14080/60000 (23%)]     Loss: 0.014489
Train Epoch: 25 [15360/60000 (26%)]     Loss: 0.018271
Train Epoch: 25 [16640/60000 (28%)]     Loss: 0.012673
Train Epoch: 25 [17920/60000 (30%)]     Loss: 0.014905
Train Epoch: 25 [19200/60000 (32%)]     Loss: 0.029832
Train Epoch: 25 [20480/60000 (34%)]     Loss: 0.016270
Train Epoch: 25 [21760/60000 (36%)]     Loss: 0.003402
Train Epoch: 25 [23040/60000 (38%)]     Loss: 0.024241
Train Epoch: 25 [24320/60000 (41%)]     Loss: 0.004467
Train Epoch: 25 [25600/60000 (43%)]     Loss: 0.048829
Train Epoch: 25 [26880/60000 (45%)]     Loss: 0.014891
Train Epoch: 25 [28160/60000 (47%)]     Loss: 0.029572
Train Epoch: 25 [29440/60000 (49%)]     Loss: 0.065137
Train Epoch: 25 [30720/60000 (51%)]     Loss: 0.039322
Train Epoch: 25 [32000/60000 (53%)]     Loss: 0.029300
Train Epoch: 25 [33280/60000 (55%)]     Loss: 0.007727
Train Epoch: 25 [34560/60000 (58%)]     Loss: 0.004799
Train Epoch: 25 [35840/60000 (60%)]     Loss: 0.011701
Train Epoch: 25 [37120/60000 (62%)]     Loss: 0.004453
Train Epoch: 25 [38400/60000 (64%)]     Loss: 0.158006
Train Epoch: 25 [39680/60000 (66%)]     Loss: 0.008052
Train Epoch: 25 [40960/60000 (68%)]     Loss: 0.007666
Train Epoch: 25 [42240/60000 (70%)]     Loss: 0.015857
Train Epoch: 25 [43520/60000 (72%)]     Loss: 0.042478
Train Epoch: 25 [44800/60000 (75%)]     Loss: 0.064638
Train Epoch: 25 [46080/60000 (77%)]     Loss: 0.022020
Train Epoch: 25 [47360/60000 (79%)]     Loss: 0.011382
Train Epoch: 25 [48640/60000 (81%)]     Loss: 0.021379
Train Epoch: 25 [49920/60000 (83%)]     Loss: 0.036568
Train Epoch: 25 [51200/60000 (85%)]     Loss: 0.068297
Train Epoch: 25 [52480/60000 (87%)]     Loss: 0.013931
Train Epoch: 25 [53760/60000 (90%)]     Loss: 0.015287
Train Epoch: 25 [55040/60000 (92%)]     Loss: 0.032512
Train Epoch: 25 [56320/60000 (94%)]     Loss: 0.034484
Train Epoch: 25 [57600/60000 (96%)]     Loss: 0.021017
Train Epoch: 25 [58880/60000 (98%)]     Loss: 0.009698

Test set: Average loss: 0.0368, Accuracy: 9894/10000 (99%)

Traning and Testing total excution time is: 570.4264826774597 seconds
```

4.1.2. Dropout in FC (File: **lenet_mnist_DO.py**)

```
Test set: Average loss: 0.0472, Accuracy: 9867/10000 (99%)

Train Epoch: 25 [0/60000 (0%)]  Loss: 0.020968
Train Epoch: 25 [1280/60000 (2%)]       Loss: 0.022872
Train Epoch: 25 [2560/60000 (4%)]       Loss: 0.034923
Train Epoch: 25 [3840/60000 (6%)]       Loss: 0.013398
Train Epoch: 25 [5120/60000 (9%)]       Loss: 0.022014
Train Epoch: 25 [6400/60000 (11%)]      Loss: 0.007233
Train Epoch: 25 [7680/60000 (13%)]      Loss: 0.035276
Train Epoch: 25 [8960/60000 (15%)]      Loss: 0.028020
Train Epoch: 25 [10240/60000 (17%)]     Loss: 0.014446
Train Epoch: 25 [11520/60000 (19%)]     Loss: 0.046610
Train Epoch: 25 [12800/60000 (21%)]     Loss: 0.012874
Train Epoch: 25 [14080/60000 (23%)]     Loss: 0.048706
Train Epoch: 25 [15360/60000 (26%)]     Loss: 0.018853
Train Epoch: 25 [16640/60000 (28%)]     Loss: 0.033166
Train Epoch: 25 [17920/60000 (30%)]     Loss: 0.026373
Train Epoch: 25 [19200/60000 (32%)]     Loss: 0.037375
Train Epoch: 25 [20480/60000 (34%)]     Loss: 0.015114
Train Epoch: 25 [21760/60000 (36%)]     Loss: 0.004222
Train Epoch: 25 [23040/60000 (38%)]     Loss: 0.057279
Train Epoch: 25 [24320/60000 (41%)]     Loss: 0.043605
Train Epoch: 25 [25600/60000 (43%)]     Loss: 0.015841
Train Epoch: 25 [26880/60000 (45%)]     Loss: 0.001968
Train Epoch: 25 [28160/60000 (47%)]     Loss: 0.061428
Train Epoch: 25 [29440/60000 (49%)]     Loss: 0.017829
Train Epoch: 25 [30720/60000 (51%)]     Loss: 0.021706
Train Epoch: 25 [32000/60000 (53%)]     Loss: 0.013850
Train Epoch: 25 [33280/60000 (55%)]     Loss: 0.002493
Train Epoch: 25 [34560/60000 (58%)]     Loss: 0.027428
Train Epoch: 25 [35840/60000 (60%)]     Loss: 0.016895
Train Epoch: 25 [37120/60000 (62%)]     Loss: 0.002574
Train Epoch: 25 [38400/60000 (64%)]     Loss: 0.114750
Train Epoch: 25 [39680/60000 (66%)]     Loss: 0.003196
Train Epoch: 25 [40960/60000 (68%)]     Loss: 0.023873
Train Epoch: 25 [42240/60000 (70%)]     Loss: 0.040449
Train Epoch: 25 [43520/60000 (72%)]     Loss: 0.026293
Train Epoch: 25 [44800/60000 (75%)]     Loss: 0.081838
Train Epoch: 25 [46080/60000 (77%)]     Loss: 0.009010
Train Epoch: 25 [47360/60000 (79%)]     Loss: 0.014812
Train Epoch: 25 [48640/60000 (81%)]     Loss: 0.015636
Train Epoch: 25 [49920/60000 (83%)]     Loss: 0.004108
Train Epoch: 25 [51200/60000 (85%)]     Loss: 0.091478
Train Epoch: 25 [52480/60000 (87%)]     Loss: 0.006550
Train Epoch: 25 [53760/60000 (90%)]     Loss: 0.007061
Train Epoch: 25 [55040/60000 (92%)]     Loss: 0.010561
Train Epoch: 25 [56320/60000 (94%)]     Loss: 0.047298
Train Epoch: 25 [57600/60000 (96%)]     Loss: 0.006503
Train Epoch: 25 [58880/60000 (98%)]     Loss: 0.023195

Test set: Average loss: 0.0342, Accuracy: 9906/10000 (99%)

Traning and Testing total excution time is: 362.6552963256836 seconds
```

4.2.3. Batch Normalization in CONV (File: **lenet_mnist_BN.py**)

```
Train Epoch: 25 [2560/60000 (4%)]       Loss: 0.006048
Train Epoch: 25 [3840/60000 (6%)]       Loss: 0.007441
Train Epoch: 25 [5120/60000 (9%)]       Loss: 0.004073
Train Epoch: 25 [6400/60000 (11%)]      Loss: 0.013845
Train Epoch: 25 [7680/60000 (13%)]      Loss: 0.009903
Train Epoch: 25 [8960/60000 (15%)]      Loss: 0.042902
Train Epoch: 25 [10240/60000 (17%)]     Loss: 0.053770
Train Epoch: 25 [11520/60000 (19%)]     Loss: 0.009015
Train Epoch: 25 [12800/60000 (21%)]     Loss: 0.004357
Train Epoch: 25 [14080/60000 (23%)]     Loss: 0.008744
Train Epoch: 25 [15360/60000 (26%)]     Loss: 0.007400
Train Epoch: 25 [16640/60000 (28%)]     Loss: 0.015072
Train Epoch: 25 [17920/60000 (30%)]     Loss: 0.028200
Train Epoch: 25 [19200/60000 (32%)]     Loss: 0.020618
Train Epoch: 25 [20480/60000 (34%)]     Loss: 0.025065
Train Epoch: 25 [21760/60000 (36%)]     Loss: 0.008998
Train Epoch: 25 [23040/60000 (38%)]     Loss: 0.005628
Train Epoch: 25 [24320/60000 (41%)]     Loss: 0.027412
Train Epoch: 25 [25600/60000 (43%)]     Loss: 0.039074
Train Epoch: 25 [26880/60000 (45%)]     Loss: 0.010339
Train Epoch: 25 [28160/60000 (47%)]     Loss: 0.018290
Train Epoch: 25 [29440/60000 (49%)]     Loss: 0.036280
Train Epoch: 25 [30720/60000 (51%)]     Loss: 0.013277
Train Epoch: 25 [32000/60000 (53%)]     Loss: 0.001687
Train Epoch: 25 [33280/60000 (55%)]     Loss: 0.010710
Train Epoch: 25 [34560/60000 (58%)]     Loss: 0.009483
Train Epoch: 25 [35840/60000 (60%)]     Loss: 0.034245
Train Epoch: 25 [37120/60000 (62%)]     Loss: 0.076052
Train Epoch: 25 [38400/60000 (64%)]     Loss: 0.005429
Train Epoch: 25 [39680/60000 (66%)]     Loss: 0.016488
Train Epoch: 25 [40960/60000 (68%)]     Loss: 0.006292
Train Epoch: 25 [42240/60000 (70%)]     Loss: 0.060219
Train Epoch: 25 [43520/60000 (72%)]     Loss: 0.001576
Train Epoch: 25 [44800/60000 (75%)]     Loss: 0.013408
Train Epoch: 25 [46080/60000 (77%)]     Loss: 0.019791
Train Epoch: 25 [47360/60000 (79%)]     Loss: 0.006758
Train Epoch: 25 [48640/60000 (81%)]     Loss: 0.009788
Train Epoch: 25 [49920/60000 (83%)]     Loss: 0.007670
Train Epoch: 25 [51200/60000 (85%)]     Loss: 0.002802
Train Epoch: 25 [52480/60000 (87%)]     Loss: 0.001018
Train Epoch: 25 [53760/60000 (90%)]     Loss: 0.059147
Train Epoch: 25 [55040/60000 (92%)]     Loss: 0.005471
Train Epoch: 25 [56320/60000 (94%)]     Loss: 0.010316
Train Epoch: 25 [57600/60000 (96%)]     Loss: 0.008525
Train Epoch: 25 [58880/60000 (98%)]     Loss: 0.007407

Test set: Average loss: 0.0294, Accuracy: 9903/10000 (99%)

Traning and Testing total excution time is: 864.2226438522339 seconds
```

4.2.4. No Dropout in FC and No Batch Normalization in CONV

(File: **lenet_mnist_No_DO_BN.py**)

```
Test set: Average loss: 0.0403, Accuracy: 9868/10000 (99%)

Train Epoch: 25 [0/60000 (0%)]  Loss: 0.020336
Train Epoch: 25 [1280/60000 (2%)]       Loss: 0.008198
Train Epoch: 25 [2560/60000 (4%)]       Loss: 0.009483
Train Epoch: 25 [3840/60000 (6%)]       Loss: 0.003530
Train Epoch: 25 [5120/60000 (9%)]       Loss: 0.015668
Train Epoch: 25 [6400/60000 (11%)]      Loss: 0.013823
Train Epoch: 25 [7680/60000 (13%)]      Loss: 0.008822
Train Epoch: 25 [8960/60000 (15%)]      Loss: 0.017715
Train Epoch: 25 [10240/60000 (17%)]     Loss: 0.005369
Train Epoch: 25 [11520/60000 (19%)]     Loss: 0.003870
Train Epoch: 25 [12800/60000 (21%)]     Loss: 0.006998
Train Epoch: 25 [14080/60000 (23%)]     Loss: 0.022801
Train Epoch: 25 [15360/60000 (26%)]     Loss: 0.002906
Train Epoch: 25 [16640/60000 (28%)]     Loss: 0.012635
Train Epoch: 25 [17920/60000 (30%)]     Loss: 0.025478
Train Epoch: 25 [19200/60000 (32%)]     Loss: 0.013117
Train Epoch: 25 [20480/60000 (34%)]     Loss: 0.026804
Train Epoch: 25 [21760/60000 (36%)]     Loss: 0.004770
Train Epoch: 25 [23040/60000 (38%)]     Loss: 0.003971
Train Epoch: 25 [24320/60000 (41%)]     Loss: 0.023954
Train Epoch: 25 [25600/60000 (43%)]     Loss: 0.019320
Train Epoch: 25 [26880/60000 (45%)]     Loss: 0.012140
Train Epoch: 25 [28160/60000 (47%)]     Loss: 0.037439
Train Epoch: 25 [29440/60000 (49%)]     Loss: 0.035279
Train Epoch: 25 [30720/60000 (51%)]     Loss: 0.033217
Train Epoch: 25 [32000/60000 (53%)]     Loss: 0.014451
Train Epoch: 25 [33280/60000 (55%)]     Loss: 0.008853
Train Epoch: 25 [34560/60000 (58%)]     Loss: 0.018058
Train Epoch: 25 [35840/60000 (60%)]     Loss: 0.009682
Train Epoch: 25 [37120/60000 (62%)]     Loss: 0.028084
Train Epoch: 25 [38400/60000 (64%)]     Loss: 0.011514
Train Epoch: 25 [39680/60000 (66%)]     Loss: 0.045906
Train Epoch: 25 [40960/60000 (68%)]     Loss: 0.004553
Train Epoch: 25 [42240/60000 (70%)]     Loss: 0.023232
Train Epoch: 25 [43520/60000 (72%)]     Loss: 0.009564
Train Epoch: 25 [44800/60000 (75%)]     Loss: 0.026771
Train Epoch: 25 [46080/60000 (77%)]     Loss: 0.017516
Train Epoch: 25 [47360/60000 (79%)]     Loss: 0.011923
Train Epoch: 25 [48640/60000 (81%)]     Loss: 0.010911
Train Epoch: 25 [49920/60000 (83%)]     Loss: 0.010205
Train Epoch: 25 [51200/60000 (85%)]     Loss: 0.015794
Train Epoch: 25 [52480/60000 (87%)]     Loss: 0.004527
Train Epoch: 25 [53760/60000 (90%)]     Loss: 0.038905
Train Epoch: 25 [55040/60000 (92%)]     Loss: 0.049313
Train Epoch: 25 [56320/60000 (94%)]     Loss: 0.003892
Train Epoch: 25 [57600/60000 (96%)]     Loss: 0.002969
Train Epoch: 25 [58880/60000 (98%)]     Loss: 0.021642

Test set: Average loss: 0.0298, Accuracy: 9909/10000 (99%)

Traning and Testing total excution time is: 452.5763108730316 seconds
```

5. Analysis

● The reason for such high accuracy for any of the variants of the Lenet-5 model implemented is only because of the dataset used.
● We have used MNIST dataset for training and testing of different Lenet-5 models
● MNIST dataset consists of images in black and white format and thus it does not have any RGB channels like CIFAR-10 images *(which was used for Homework 4).*
● Since, MNIST dataset has single channel black and white images, the model trains very well and hence the accuracy is high i.e., **99** %.
● This was not the case with CIFAR-10 dataset images where the accuracy was between 50-65% for Lenet-5 based models which is quite low with respect to MNIST dataset.

6. Machine Specifications

- Google Colab

7. Conclusion

As per the performance analysis, model with only dropout layer in the model takes the least time. Simple Lenet-5 model where NO dropout and NO batch normalization layers are present in model, takes the second least time to train and test followed by variant 1 i.e., model with dropout and batch normalization layers in model further followed by variant 3 i.e., model with only batch normalization layer which comes last and takes the most time for training and testing. Also, we discussed the reason for high accuracy across all the models which was due to MNIST dataset having 1 channel dataset i.e., only black, and white images and also some analysis on the accuracy of models after the first epoch.

8. References

1. https://bdtechtalks.com/2020/01/06/convolutional-neural-networks-cnn-convnets/
2. https://medium.com/@amarbudhiraja/https-medium-com-amarbudhiraja-learning-less-to-learn-better-dropout-in-deep-machine-learning-74334da4bfc5
3. https://machinelearningmastery.com/batch-normalization-for-training-of-deep-neural-networks/